Exercise 1: USART Communication (Microchip Studio, Arduino MEGA)

1 Introduction

In this exercise you will use the USART peripheral to communicate between your Arduino MEGA and your computer. To complete the task, you only need the Arduino MEGA and PuTTY.

If you have not installed PuTTY yet, refer to slide 11 in Basic Microchip Studio.pdf

2 Useful References

- ATmega640/1280/1281/2560/2561 Datasheet
- Understanding the USART Baud Rate Macro
- AVR LIBC manual
- Simple Serial Communications With AVR LIBC (blogpost) by Mika Tuupola

3 A note on terminology

When you are interfacing with hardware in embedded systems you often hear term **register**. This term might be confusing because it means two things that are related but not exactly the same. Overall the register is a unit of memory that hardware can operate with.

For processor or microcontroller cores registers are cells that store values loaded from general purpose memory or peripherals and between which the core can compute some operations producing computation result that is then also stored into one of these registers.

For peripheral hardware such as serial ports or general purpose input-output (GPIO) peripherals the register is a cell in the hardware logic that controls the operation of hardware. For example an 8-bit wide cell of GPIO might be used to define states of 8 output pins. The communication between processor core and peripheral hardware happens on communication buses. Each microprocessor system might have one or multiple buses that have different parameters. The important part is that in order to access data on a bus most systems give each hardware unit a set of unique addresses **mapping** every register that unit has. This process is confusingly called **memory-mapping** it does not mean, however, that device register gets some memory allocated for it. Instead the device register is a memory by itself. The memory ranges in which device registers are mapped are usually not continuous and accessing invalid values will trigger fault states of different kind. The general purpose memory can also be seen as one of the peripherals with a large address range and no **side-effects** when that memory is written or read. Here **side-effect** is some behavior in real world. E.g. writing to GPIO output register will set pin outputs to new values which will reflect in changed voltages on microcontroller pins whereas writing to general purpose memory will do nothing except for storing that data for later retrieval.

4 Skeleton code

You are provided with a minimal application that uses serial port to query user name and send Hello, username! message every second. Include the provided files into your Microchip studio project. See slides 5 and 10 of Basic Microchip Studio.pdf on how to create an empty project and add library files.

The following files are provided:

- main.c - contains entry point and core logic of the application.
- mcu.h - contains constants defined for MCU core such as processor clock speed.
- uart.h - contains configuration constants and definition of public interface for serial peripheral (UART) driver.

Upon boot the application will execute `void setup_uart_io(void)` of UART driver to let the driver execute necessary hardware and software configuration. After that all interaction with serial port driver is handled via standard library IO functions that will redirect any writes to **stdin** and reads from **stdout** to serial port driver. Note that this example leaves **stderr** unconfigured.

5 Exercise task

Please review the skeleton code and comments carefully. Ask questions if some parts are not clear. Answer the following questions:

- What is the configured clock speed of MCU?
- What is the configured serial port baud rate?

Write serial driver module **uart.c** that implements public driver interface defined in uart.h and allows normal operation of **stdin** and **stdout** IO streams with LIBC functions.

6 Includes

Add required `#include`s to define clock speeds and baud rates:

```
#include "mcu.h"
#include "uart.h"

#include <avr/io.h>
#include <stdio.h>
```

7 Clocks and prescalers

Every microcontroller system has at least one master clock that drives logic of the core. The peripheral hardware logic however often operates at **clock rate** (clock frequency) different (and usually lower) than the master clock rate. To achieve synchronized operation and produce low-rate clock for each purpose the microcontrollers produce derived clocks. One simple way to produce derived clock is to have a simple counter that decrements with the main clock rate and scaler register that defines the initial value that timer. When timer reaches 0 it is **reloaded** to the initial value and starts counting again. Every time the counter is reloaded the derived clock **toggles** between HIGH and LOW states. This produces a clock that is related to the main clock by is $2N$ times slower than the main clock where N is the divider value.

8 Baud Rate Formula

The USART peripheral typically needs to output its data at a low rate. To produce the clock used by USART the Baud Rate Register (UBRR) is used with its value treated as reload value of the clock dividing counter. Its value is computed as:

$$\text{UBRR} = \frac{F_{\text{OSC}}}{16 \cdot \text{BAUD}} - 1$$

You can also find specific UBRR values in datasheet Table 22-12 (p. 226). Note here that BAUD rate unit is baud (which in case of UART are equivalent to bits per second). The additional division of input clock by 16 is described in datasheet Figure 22-2. Clock Generation Logic, Block Diagram that presents 3 additional fixed dividers labeled "/2" (meaning dividing clock by constant factor of 2) and "/4" (constant divider with a factor of 4). E.g., the fastest baud rate this MCU can produce at 16 MHz main clock as 1000000 bits/s when UBRR is set to 0.

To implement the calculation of UBRR add the following after the list of includes:

```
#define MYUBRR (F_CPU/16/UART_BAUD_RATE-1) ///< Define the UBRR value using
    the equation in datasheet p.203 Table 22-1
```

9 USART driver functions

9.1 Initialization Function

This function sets up UART registers. You can find explanation of values for each register in datasheet 22.10 Register Description pp. 218-222 Note these registers look and act like normal C variables but we have not defined them anywhere. Instead they come from `#include <avr/io.h>` header. Each value points to a single address and the type of the variable is typically `uint8_t` (8-bit wide unsigned integer) for AVR 8-bit MCUs. However, some registers might be 16 or 32-bit wide. The datasheet provides the information on size of each variable. These values are specific to the configured microcontroller chip. If you specify wrong chip in project configuration some of these constant might contain invalid values resulting in broken firmware.

```
// NOTE: USART = Universal Synchronous Asynchronous Receiver Transmitter
//       UART  = Universal Asynchronous Receiver Transmitter

static void USART_init(uint16_t ubrr) //USART initiation p.206
{
    /* Set baud rate in the USART Baud Rate Registers (UBRR) *///datasheet p.222
    UBRR0H = (unsigned char) (ubrr >> 8); //datasheet p.206, ubbr value is 103
    UBRR0L = (unsigned char) ubrr; //datasheet p.206, , ubbr value is 103

    /* Enable receiver and transmitter on RX0 and TX0 */
    UCSR0B |= (1 << RXEN0) | (1 << TXEN0); //RX complete interrupt enable//Transmitter enable // datasheet p.206, p.220

    /* Set frame format: 8 bit data, 2 stop bit */
    UCSR0C |= (1 << USBS0) | (3 << UCSZ00);//Stop bit selection at 2-bit//UCSZ bit setting at 8 bit//datasheet p.221 and p.222
}
```

9.2 USART Transmission

This function is called by AVR LIBC when a character is written to stdout. The FILE *stream argument is not used because we only have one place to output that byte.

This function is a simple **blocking** implementation which means it will waste processor time until the hardware is ready to send the data.

```
static int USART_Transmit(char data, FILE *stream) //datasheet p.207
{
    /* Wait until the transmit buffer is empty*/
    while(!(UCSR0A & (1 << UDRE0))) //datasheet p.207, p. 219
    {
        ;
    }

    /* Puts the data into a buffer, then sends/transmits the data */
    UDR0 = data;

    return 0;
}
```

9.3 USART Reception

This function is called by AVR LIBC when a character needs to be read from stdout. The FILE *stream argument is not used because we only have one place to read that byte from.

This function is a simple **blocking** implementation which means it will **waste processor time until there is some data to be read**. It is not easy to add timeout here because it will not be handled by AVR LIBC.

```
static int USART_Receive(FILE *stream)
{
    // Wait until data is received
    while (!(UCSR0A & (1 << RXC0))) { }

    // Return received byte
    return UDR0;
}
```

9.4 Setting up IO streams

Then we need to use ¸FDEV_SETUP_STREAM macro functions from AVR LIBC to setup IO streams that will be later assigned to **stdin** and **stdout**.

```
FILE uart_output = FDEV_SETUP_STREAM(USART_Transmit, NULL, _FDEV_SETUP_WRITE);
FILE uart_input  = FDEV_SETUP_STREAM(NULL, USART_Receive, _FDEV_SETUP_READ);
```

9.5 Driver's public interface

This is the entry point for configuration of our driver. This function should be called only once after which normal IO functions like printf and scanf can be used in the rest of the program.

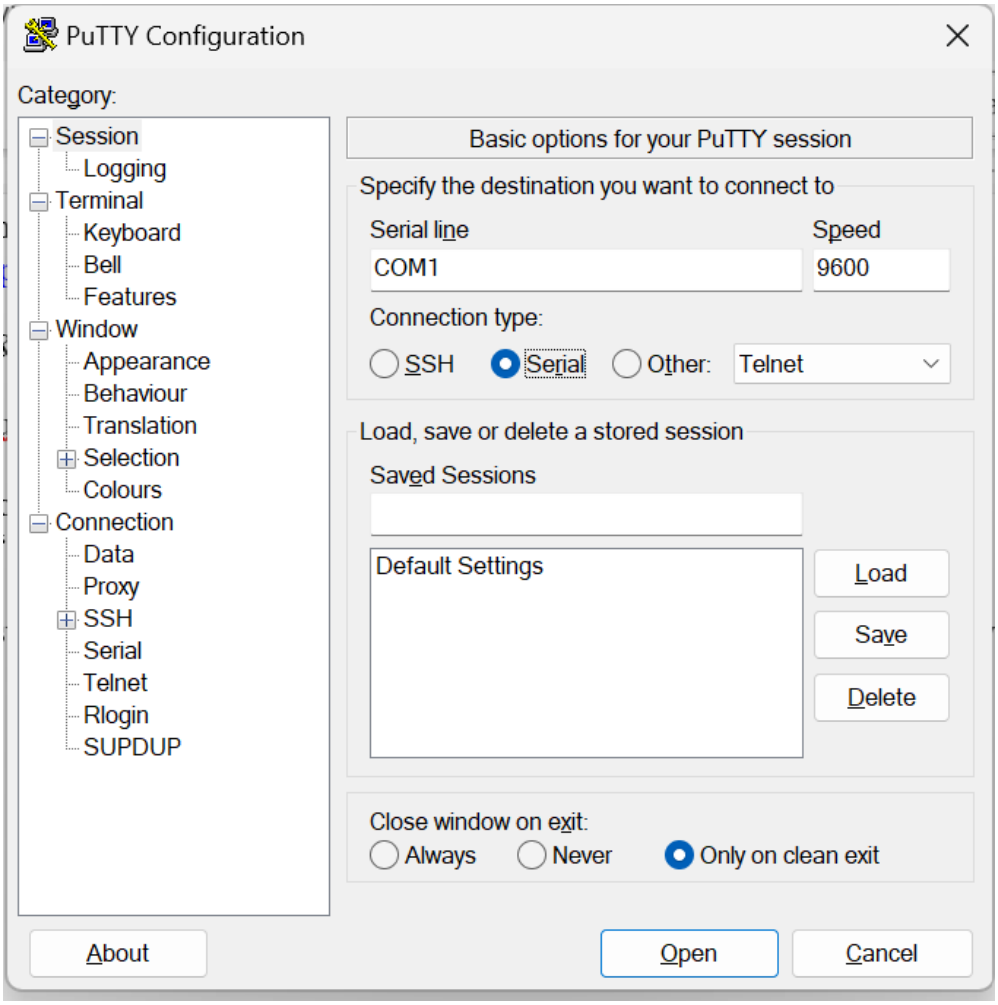
```
uint8_t setup_uart_io(void)
{
    // initialize the UART with 9600 BAUD. Here we pass not the desired baud
       rate but an already configured scaler value.
    USART_init(MYUBRR);

    // redirect the stdin and stdout to UART functions,  read  https://
       appelsiini.net/2011/simple-usart-with-avr-libc/
    stdout = &uart_output;
    stdin = &uart_input;

    return 0;
}
```

10 Using PuTTY

1. Build and upload your firmware in Microchip Studio.
2. Open PuTTY.
3. Select connection type: **Serial**.
4. Set the correct **COM port**.
5. Set **Speed** = **9600**.
6. Click **Open**.


PuTTY Configuration
Category:
Session
Logging
Terminal
Keyboard
Bell
Features
Window
Appearance
Behaviour
Translation
Selection
Colours
Connection
Data
Proxy
SSH
Serial
Telnet
Rlogin
SUPDUP
Basic options for your PuTTY session
Specify the destination you want to connect to
Serial line
Speed
COM1
9600
Connection type:
SSH
Serial
Other:
Telnet
Load, save or delete a stored session
Saved Sessions
Default Settings
Load
Save
Delete
Close window on exit:
Always
Never
Only on clean exit
About
Open
Cancel